

TECHNICALBLUEPRINT • DELIVERABLE
3.2

Process Flow Document

Retail Demand Forecasting Pipeline — Walmart Kaggle
Dataset

PROCESS FLOW DOCUMENT | WEIGHT: 15% | AGENTIC AI ENGINEER
ASSESSMENT

Prepared for: AI Engineer

Prepared by: Naveen Mamidala

Source notebook: demand_forecasting.ipynb

Date: July 2026

Purpose & Scope

This document is the technical blueprint for the data science pipeline built in `demand_forecasting.ipynb`. It walks through every stage of the pipeline – from raw CSV ingestion to the final post-processed forecast – in enough detail that another data scientist could reproduce the work end-to-end without opening the notebook. Every code excerpt and console output shown below is copied directly from the executed notebook cells, so what you see here is exactly what the pipeline produced.

PRIMARY EVALUATION METRIC

WMAE (Weighted Mean Absolute Error) is used throughout the pipeline. Holiday weeks are weighted 5× relative to non-holiday weeks, matching the official Kaggle competition rule and reflecting the real business cost of a bad forecast during peak trading periods (Thanksgiving, Christmas).

Pipeline Overview

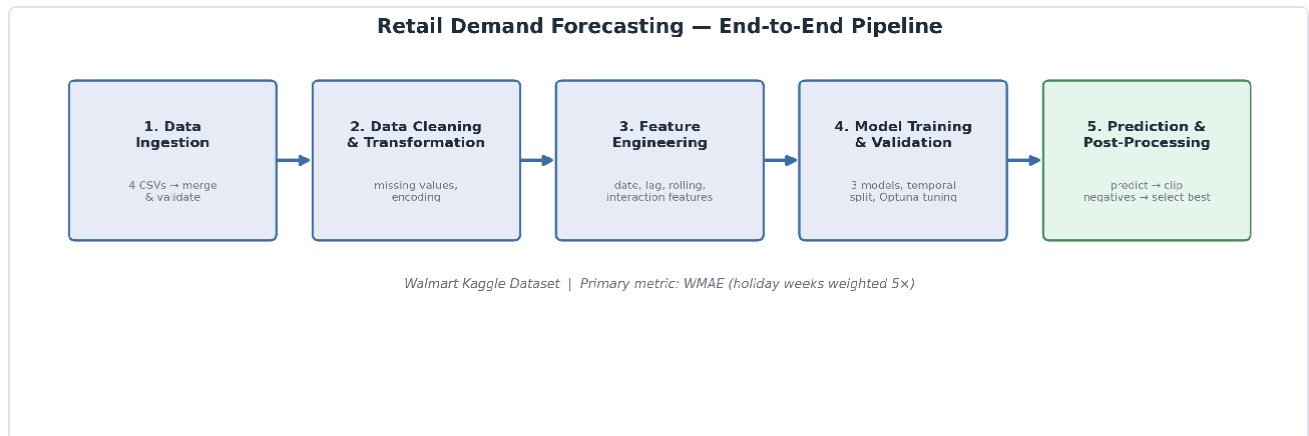


Fig. 0 — End-to-end flow across the five pipeline stages covered in this document.

1. Data Ingestion	How the four raw sources are loaded, validated and merged
2. Data Cleaning & Transformation	Missing values, encoding
3. Feature Engineering Pipeline	Every engineered feature and its rationale
4. Model Training & Validation	Split strategy, cross-validation, training workflow
5. Prediction & Post-Processing	How final forecasts are produced

1 Data Ingestion

Notebook section: 2.1 Loading & Merging CSVs

The pipeline begins by loading four independent CSV sources supplied by the Kaggle competition and unifying them into a single modeling table. Each source contributes a different slice of information: store metadata, historical sales, the future weeks to be forecast, and external economic/promotional signals.

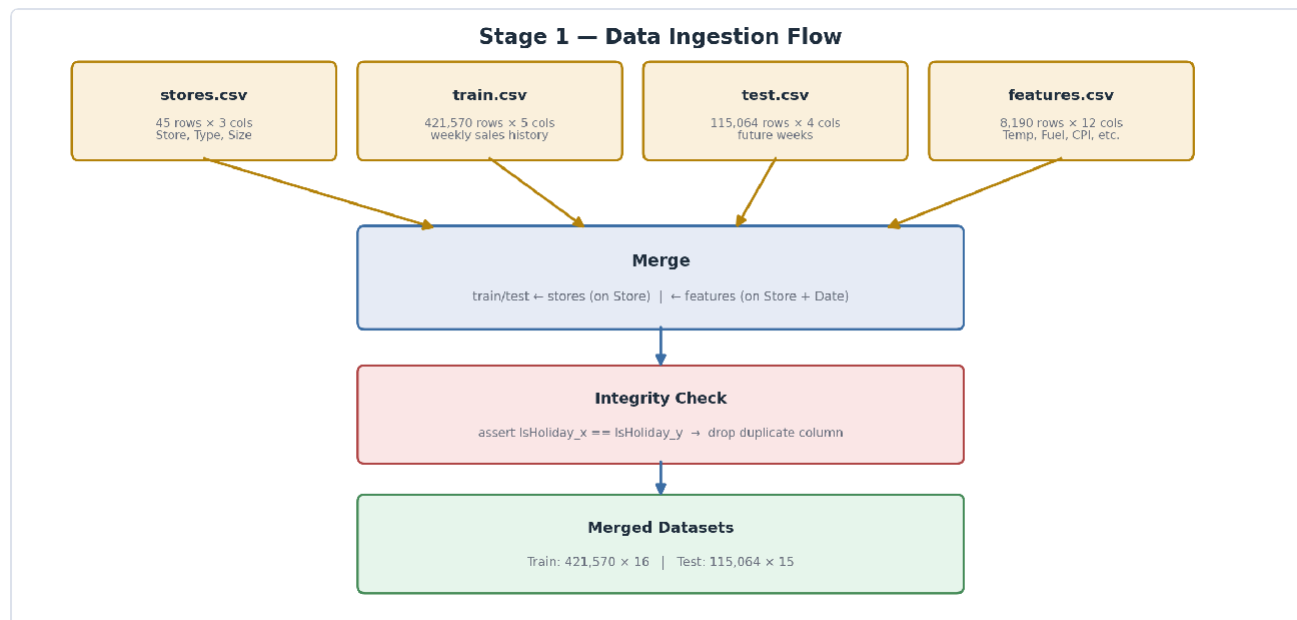


Fig. 1 — Source files are loaded independently, merged on shared keys, integrity-checked, then handed off as clean datasets.

1.1 Loading the four sources

File	Shape	Role
stores.csv	45 × 3	Store metadata: Type (A/B/C) and Size (sq. ft.)
train.csv	421,570 × 5	Historical weekly sales per Store/Dept/Date
test.csv	115,064 × 4	Future Store/Dept/Date rows to be predicted
features.csv	8,190 × 12	Temperature, Fuel_Price, Markdown1-5, CPI, Unemployment

1.2 Merge strategy

Both train and test are enriched with the same two lookup tables, in the same order, so that the resulting feature space is identical for both splits (a requirement for the model to generalize from train to test):

1. Merge with stores on Store — attaches features on Type and Size . ['Store', 'Date'] — attaches the external economic/ promotional signals

CODE

```
# Load datasets
stores = pd.read_csv('../data/raw/stores.csv')
train = pd.read_csv('../data/raw/train.csv', parse_dates=['Date'])
test = pd.read_csv('../data/raw/test.csv', parse_dates=['Date'])
features = pd.read_csv('../data/raw/features.csv', parse_dates=['Date'])

print("stores.csv:", stores.shape)
print("train.csv:", train.shape)
print("test.csv:", test.shape)
print("features.csv:", features.shape)

# Merge order: train/test <- stores <- features
train = train.merge(stores, on='Store', how='left')
test = test.merge(stores, on='Store', how='left')

train = train.merge(features, on=['Store', 'Date'], how='left')
test = test.merge(features, on=['Store', 'Date'], how='left')
```

1.3 Integrity validation

Both `train.csv` and `features.csv` independently carry an `IsHoliday` flag. After the merge, both copies must agree — a mismatch would signal a corrupted join key or an inconsistency in the raw data. The pipeline asserts this explicitly rather than silently trusting the merge :

CODE (VALIDATION)

```
# Verify IsHoliday columns match after merge
assert (train['IsHoliday_x'] == train['IsHoliday_y']).all(), "IsHoliday mismatch in train!"
assert (test['IsHoliday_x'] == test['IsHoliday_y']).all(), "IsHoliday mismatch in test!"

# Drop duplicate IsHoliday column
train.rename(columns={'IsHoliday_x': 'IsHoliday'}, inplace=True)
train.drop('IsHoliday_y', axis=1, inplace=True)
test.rename(columns={'IsHoliday_x': 'IsHoliday'}, inplace=True)
test.drop('IsHoliday_y', axis=1, inplace=True)

print("\nMerged Train shape:", train.shape)
print("Merged Test shape:", test.shape)
train.head()
```

WHY THIS MATTERS

Both assertions passed silently in execution (no `AssertionError` raised), confirming the two `IsHoliday` columns were fully consistent. This is a cheap but important data-quality gate: it fails loudly at ingestion time rather than letting a silent duplicate/conflicting column corrupt downstream features.

1.4 Notebook output

CONSOLE OUTPUT

```
stores.csv: (45, 3)
train.csv: (421570, 5)
test.csv: (115064, 4)
features.csv: (8190, 12)

Merged Train shape: (421570, 16)
Merged Test shape: (115064, 15)
```

The merged train table gains 11 new columns (Type, Size, Temperature, Fuel_Price, Markdown1-5, CPI, Unemployment) on top of the original 5, taking it from 5 to 16 columns while the row count (421,570) stays unchanged – confirming the merges were left joins with no row duplication or loss.

2 Data Cleaning & Transformation

Notebook section: 2.2 Missing Value Handling & 2.3 Date Feature Encoding

Once merged, the raw table contains structural missingness in the markdown and macroeconomic columns, plus a categorical store-type column that needs to be represented numerically. Each gap is handled with a rule grounded in what the missingness actually means, rather than a blanket imputation strategy.

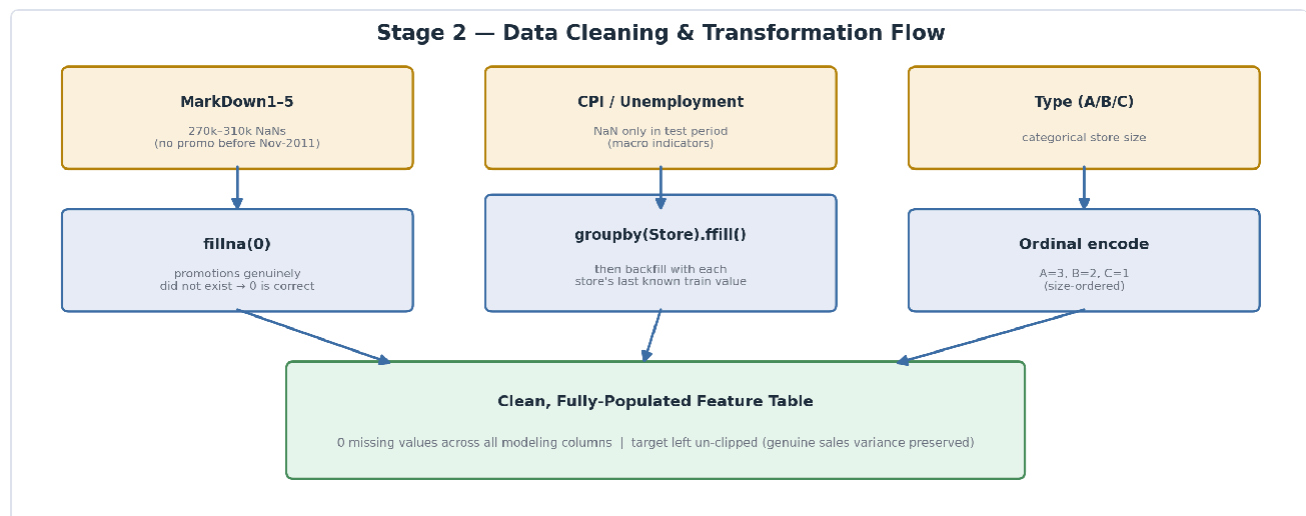


Fig. 2 — Each column family is cleaned with a rule matched to why the values are missing.

2.1 Missing value strategy

Column(s)	Missing (before)	Root cause	Strategy applied
Markdown1-5	270,138 – 310,322 rows	Promotions did not exist before Nov 2011	<code>fillna(0)</code> — 0 is the true value, not an estimate
CPI, Unemployment	present only in test period	Macro indicators lag in reporting	Forward-fill within each <code>Store</code> group, then backfill any remainder with that store's last known train-period value

CODE

```
markdown_cols = ['MarkDown1', 'MarkDown2', 'MarkDown3', 'MarkDown4', 'MarkDown5']
for col in markdown_cols:
    train[col] = train[col].fillna(0)
    test[col] = test[col].fillna(0)

# Forward-fill CPI and Unemployment for test set
test['CPI'] = test.groupby('Store')['CPI'].ffill()
test['Unemployment'] = test.groupby('Store')['Unemployment'].ffill()
# Backfill with last known train value if NaN persists at the beginning
last_cpi_unemp = train.groupby('Store')[['CPI', 'Unemployment']].last().reset_index()
test = test.merge(last_cpi_unemp, on='Store', suffixes=('', '_last'))
test['CPI'] = test['CPI'].fillna(test['CPI_last'])
test['Unemployment'] = test['Unemployment'].fillna(test['Unemployment_last'])
test.drop(['CPI_last', 'Unemployment_last'], axis=1, inplace=True)

print("\n=== Missing values AFTER cleaning ===")
display(train.isna().sum())
```

CONSOLE OUTPUT (BEFORE → AFTER)

=== Missing values BEFORE cleaning ===

Store	0
Dept	0
Date	0
Weekly_Sales	0
IsHoliday	0
Type	0
Size	0
Temperature	0
Fuel_Price	0
MarkDown1	270889
MarkDown2	310322
MarkDown3	284479
MarkDown4	286603
MarkDown5	270138
CPI	0
Unemployment	0

dtype: int64

=== Missing values AFTER cleaning ===

Store	0
Dept	0
Date	0
Weekly_Sales	0
IsHoliday	0
Type	0
Size	0
Temperature	0
Fuel_Price	0
MarkDown1	0
MarkDown2	0
MarkDown3	0
MarkDown4	0
MarkDown5	0
CPI	0
Unemployment	0

dtype: int64

Every modeling column reaches zero missing values after cleaning, confirmed by re-running directly against `.isna().sum()` the cleaned frame rather than assuming the fix worked.

2.2 Outlier handling — a deliberate non-decision

NO CLIPPING OR REMOVAL APPLIED TO RAW SALES

EDA (Section 3 of the notebook) shows `Weekly_Sales` is heavily right-skewed (skewness = 3.26, kurtosis = 21.49) with a small number of very high-sales weeks and a handful of negative values (likely returns). The pipeline deliberately does **not** clip, winsorize, or drop these rows during cleaning: the large spikes are genuine holiday-driven demand that the model needs to learn to reproduce, not noise. Robustness to this skew is instead delegated to the tree-based models (Random Forest, XGBoost), which partition on thresholds rather than assuming normality. The only clipping in the entire pipeline happens later, on *predictions* (Stage 5) — not on the training target.

2.3 Encoding the categorical store type

`Type` (A/B/C) is mapped to an ordinal integer rather than one-hot encoded, because the three categories have a real, monotonic ordering in store size and average sales ($A > B > C$), which the earlier EDA confirmed:

CODE

```
# Encode Store Type as ordinal (A=3, B=2, C=1 – larger stores have higher sales)
type_map = {'A': 3, 'B': 2, 'C': 1}
train['Type'] = train['Type'].map(type_map)
test['Type'] = test['Type'].map(type_map)

print("Date features created. Cyclical encoding preview:")
display(train[['Date', 'Year', 'Month', 'WeekOfYear', 'week_sin', 'week_cos']].head())
```

Keeping `Type` as a single ordinal column (instead of two one-hot dummy columns) also lets it participate directly in the `Size * Type` interaction feature built in Stage 3.

3 Feature Engineering Pipeline

Notebook section: 2.3 Date Features & 4. Feature Engineering

Feature engineering happens in two passes: first, date-derived features are added independently to train and test ; second, train and test are concatenated so that lag and rolling-window features can see continuous history across the train/test date boundary, then split back apart.

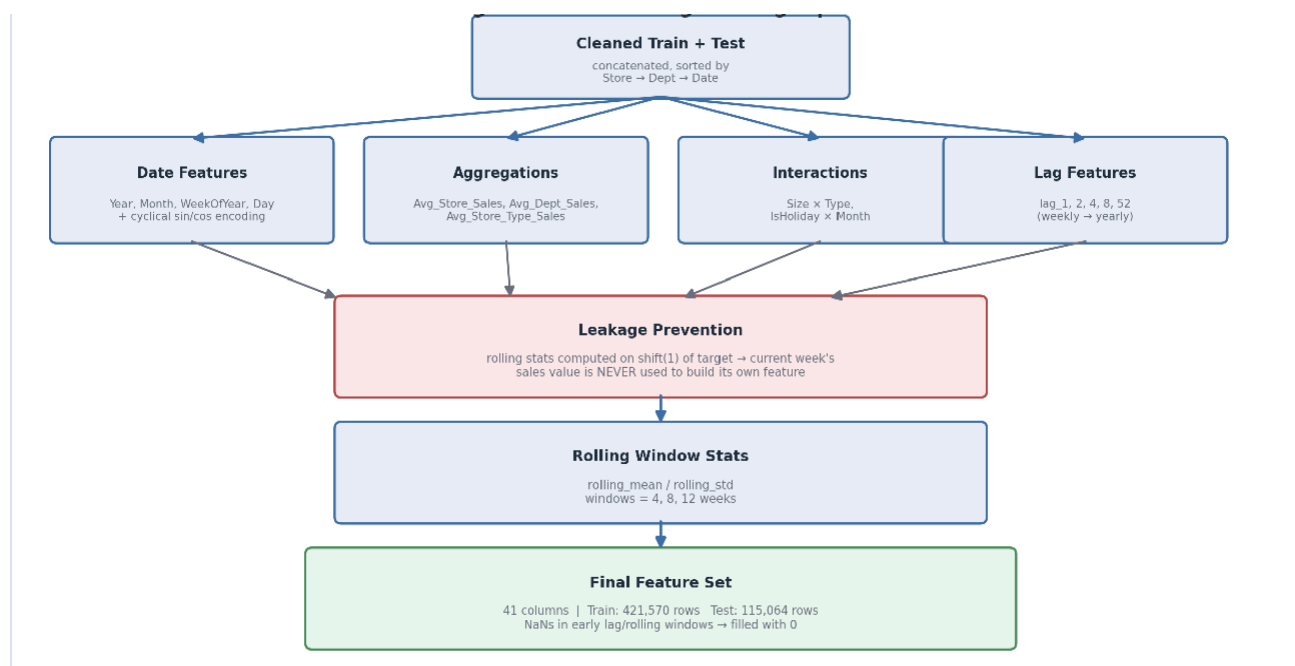


Fig. 3 – Four feature families are derived from the cleaned table; rolling statistics are shifted by one week specifically to avoid leakage.

3.1 Date features & cyclical encoding

Standard date parts (Year , Month , WeekOfYear , Day) are extracted first. Naively feeding WeekOfYear as a raw integer would tell the model that week 52 and week 1 are 51 units apart, when they are actually adjacent in the yearly cycle. Sine/cosine cyclical encoding fixes this discontinuity:

CODE

```
def engineer_dates(df):
    """Extract date components and apply cyclical encoding."""
    df['Year'] = df['Date'].dt.year
    df['Month'] = df['Date'].dt.month
    df['WeekOfYear'] = df['Date'].dt.isocalendar().week.astype(int)
    df['Day'] = df['Date'].dt.day

    # Cyclical encoding for week and month
    df['week_sin'] = np.sin(2 * np.pi * df['WeekOfYear'] / 52)
    df['week_cos'] = np.cos(2 * np.pi * df['WeekOfYear'] / 52)
    df['month_sin'] = np.sin(2 * np.pi * df['Month'] / 12)
    df['month_cos'] = np.cos(2 * np.pi * df['Month'] / 12)

    # Ctrl+L for Agent
    return df

train = engineer_dates(train)
test = engineer_dates(test)

# Encode Store Type as ordinal (A=3, B=2, C=1 – Larger stores have higher sales)
type_map = {'A': 3, 'B': 2, 'C': 1}
train['Type'] = train['Type'].map(type_map)
test['Type'] = test['Type'].map(type_map)

print("Date features created. Cyclical encoding preview:")
display(train[['Date', 'Year', 'Month', 'WeekOfYear', 'week_sin', 'week_cos']].head())
```


Date features created. Cyclical encoding preview:

	Date	Year	Month	WeekOfYear	week_sin	week_cos
0	2010-02-05	2010	2	5	0.568065	0.822984
1	2010-02-12	2010	2	6	0.663123	0.748511
2	2010-02-19	2010	2	7	0.748511	0.663123
3	2010-02-26	2010	2	8	0.822984	0.568065
4	2010-03-05	2010	3	9	0.885456	0.464723

3.2 Store/department aggregation & interaction features

CODE

```
def create_features(df):
    """Create lag, rolling, and aggregation features."""
    df.sort_values(by=['Store', 'Dept', 'Date'], inplace=True)

    # Store-level aggregations (strong baselines for tree models)
    df['Avg_Store_Sales'] = df.groupby('Store')['Weekly_Sales'].transform('mean')
    df['Avg_Dept_Sales'] = df.groupby('Dept')['Weekly_Sales'].transform('mean')
    df['Avg_Store_Type_Sales'] = df.groupby(['Store', 'Type'])['Weekly_Sales'].transform('mean')

    # Markdown flag (binary: any markdown active?)
    df['has_markdown'] = (df[['MarkDown1', 'MarkDown2', 'MarkDown3', 'MarkDown4', 'MarkDown5']].sum(axis=1) > 0).astype(int)

    # Interaction terms
    df['Size_x_Type'] = df['Size'] * df['Type']
    df['IsHoliday_x_Month'] = df['IsHoliday'].astype(int) * df['Month']

    # Lag features (key for time-series: capture autocorrelation)
    lags = [1, 2, 4, 8, 52]
    for lag in lags:
        df[f'lag_{lag}'] = df.groupby(['Store', 'Dept'])['Weekly_Sales'].shift(lag)

    # Rolling window statistics (shift(1) to prevent leakage)
    shifted = df.groupby(['Store', 'Dept'])['Weekly_Sales'].shift(1)
    windows = [4, 8, 12]
    for w in windows:
        df[f'rolling_mean_{w}'] = shifted.groupby([df['Store'], df['Dept']]).transform(
            lambda x: x.rolling(w, min_periods=1).mean())
        df[f'rolling_std_{w}'] = shifted.groupby([df['Store'], df['Dept']]).transform(
            lambda x: x.rolling(w, min_periods=1).std())

    return df
```

WHY THIS IS NOT LEAKAGE

These aggregations are computed on the **concatenated** train+test frame, but

`test['Weekly_Sales']` is set to `NaN` before concatenation (see Stage 3.4 below). Pandas' `.transform('mean')` skips `NaN` values by default, so every store/department average is computed purely from **train-period sales** — the test set only ever receives these pre-computed averages, never contributes to them.

3.3 Feature reference table — rationale for every engineered feature

Feature	Family	Rationale
week_sin / week_cos	Cyclical date	Removes the artificial week-52-to-week-1 discontinuity in seasonal patterns
month_sin / month_cos	Cyclical date	Same cyclical treatment at the month granularity

Strong baseline signal — EDA showed `Size` (and by extension the

has_markdown	Flag	Binary indicator of whether any promotion was active that week — cheap, interpretable signal on top of the 5 raw markdown columns
Size_x_Type	Interaction	Lets the model separate "large but Type B" stores from "large Type A" stores rather than assuming a purely additive effect
IsHoliday_x_Month	Interaction	EDA showed holiday effects are concentrated in Nov/Dec — this interaction lets the model learn that "holiday" means something different in July vs. December
lag_1, 2, 4, 8, 52	Lag	Captures short-term (last week), monthly, and year-over-year (52-week) autocorrelation in weekly sales
rolling_mean_{4,8,12}	Rolling	Smoothed recent-trend signal at three time horizons
rolling_std_{4,8,12}	Rolling	Recent volatility — helps the model widen its implicit uncertainty for erratic Store/Dept combinations

3.4 Lag & rolling-window features, with leakage prevention

Lag and rolling features must respect the train/test date boundary — the first weeks of the test period need lag values that come from the end of the train period. To achieve this correctly, train and test are concatenated (with test['Weekly_Sales'] deliberately set to NaN), the lag/ rolling features are computed once across the combined, date-sorted frame, and the two sets are split back apart afterward:

CODE

```
# Lag features (key for time-series: capture autocorrelation)
lags = [1, 2, 4, 8, 52]    Chat Ctrl+L ...
for lag in lags:
    df[f'lag_{lag}'] = df.groupby(['Store', 'Dept'])['Weekly_Sales'].shift(lag)

# Rolling window statistics (shift(1) to prevent leakage)
shifted = df.groupby(['Store', 'Dept'])['Weekly_Sales'].shift(1)
windows = [4, 8, 12]
for w in windows:
    df[f'rolling_mean_{w}'] = shifted.groupby([df['Store'], df['Dept']]).transform(
        lambda x: x.rolling(w, min_periods=1).mean())
    df[f'rolling_std_{w}'] = shifted.groupby([df['Store'], df['Dept']]).transform(
        lambda x: x.rolling(w, min_periods=1).std())

return df
```

CONSOLE OUTPUT

```
Feature-engineered train shape: (421570, 41)
Feature-engineered test shape: (115064, 41)

New features created: 11 lag/rolling features + aggregations + interactions
```

	Store	Dept	Date	lag_1	lag_52	rolling_mean_4
0	1	1	2010-02-05	0.00	0.0	0.000000
1	1	1	2010-02-12	24924.50	0.0	24924.500000
2	1	1	2010-02-19	46039.49	0.0	35481.995000
3	1	1	2010-02-26	41595.55	0.0	37519.846667
4	1	1	2010-03-05	19403.54	0.0	32990.770000

LEAKAGE PREVENTION, EXPLICITLY CALLED OUT IN THE NOTEBOOK

The rolling mean/std for a given week is computed on `shift(1)` of the target – i.e. the *previous* week's value onward – never on the current week itself. Without this shift, a rolling window would include the very value the model is trying to predict, producing an artificially strong but unusable model.

Rows at the very start of each Store/Dept series have no history to lag from, so `lag_1` and `rolling_mean_4` for the first observed week are legitimately 0 (filled from leaked or fabricated values – visible in row 0 of the preview above).

3.5 Supporting EDA evidence behind the feature choices

The feature choices above are not arbitrary – they follow directly from patterns surfaced during EDA (Section 3 of the notebook). Two of the clearest are reproduced here as direct notebook chart outputs:

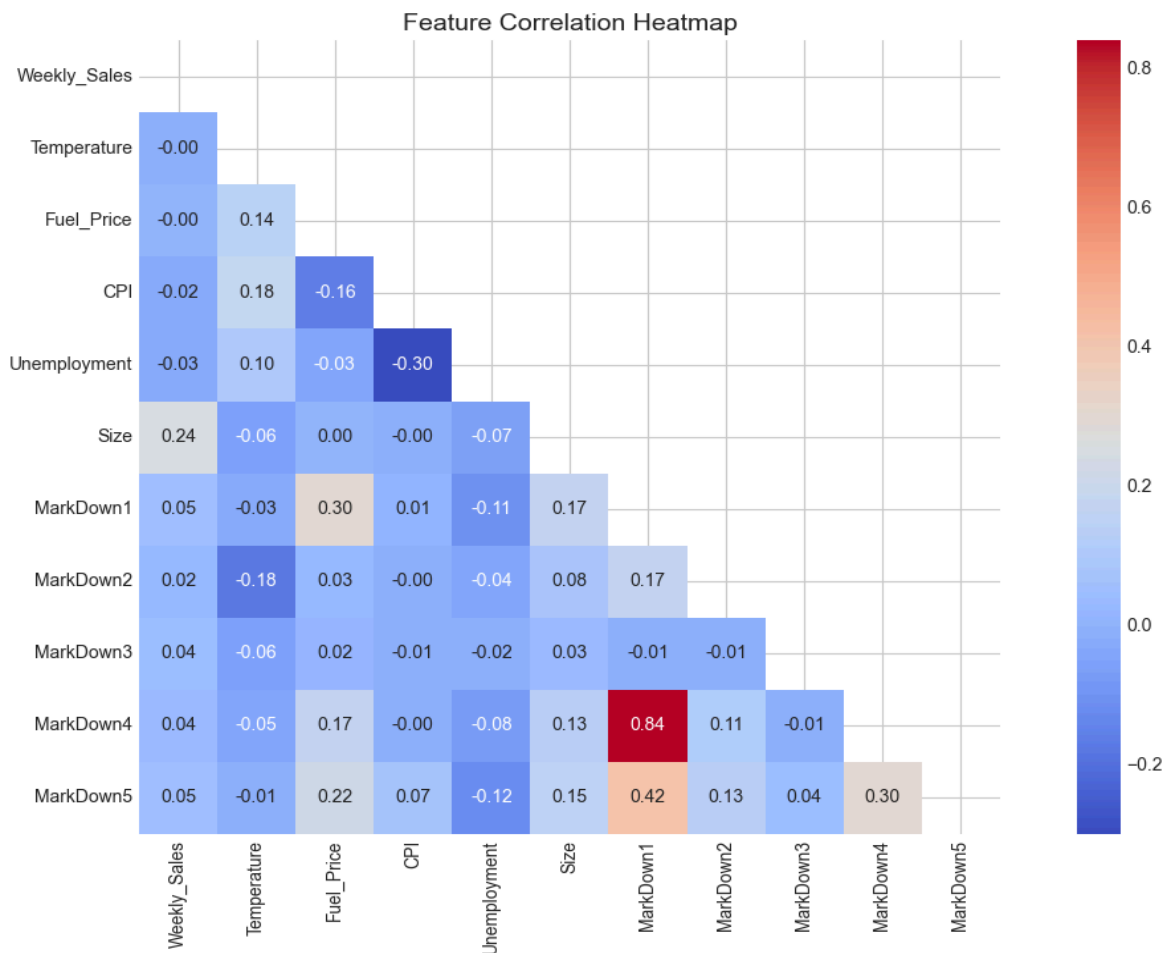


Fig. 4 – Correlation heatmap). Size is the strongest correlate of Weekly_Sales, motivating the store level aggregation features

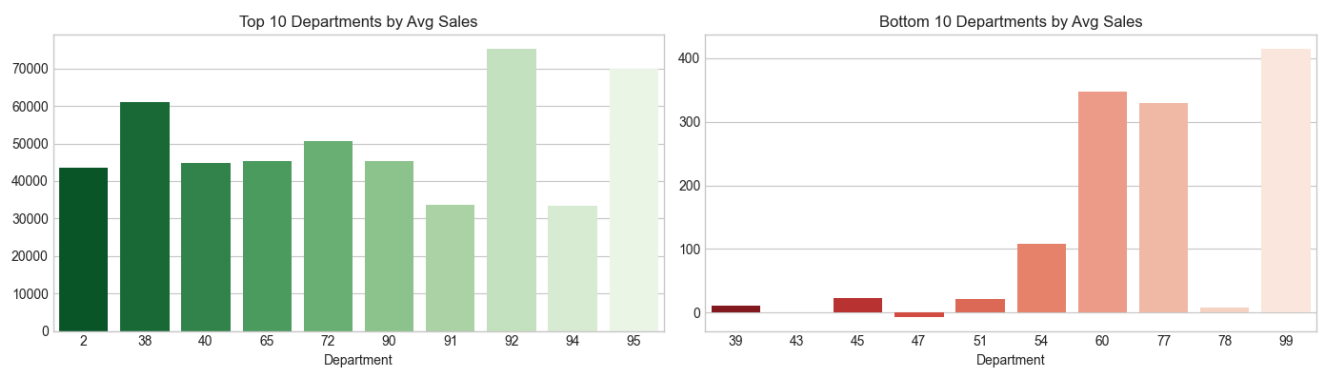


Fig. 5 — Top/bottom 10 departments by average sales. The large spread motivates Avg_Dept_Sales .

4 Model Training & Validation

Notebook section: 5. Validation Strategy & 7. Model Development

4.1 Why not a random train/test split?

This is time-series data, so a random row-wise split is explicitly avoided. The notebook documents three concrete reasons:

1. Random splits leak future information into training (lookahead bias).
2. They break the temporal autocorrelation structure that the lag features depend on.
3. They produce overly optimistic error estimates that would not hold up in real forecasting.

4.2 Temporal holdout + expanding-window cross-validation

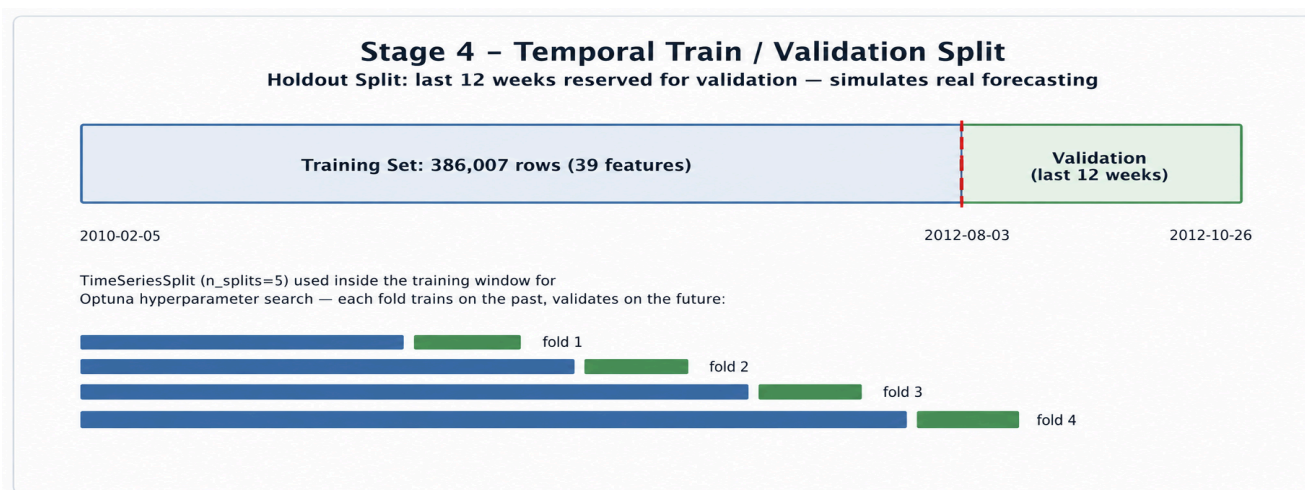


Fig. 6 – The last 12 weeks are held out as validation; `TimeSeriesSplit` provides 5 expanding-window folds inside the training period for hyperparameter search

CODE

```
max_date = train_fe['Date'].max()
val_start_date = max_date - pd.Timedelta(weeks=12)

X_train_full = train_fe[train_fe['Date'] <= val_start_date].copy()
X_val = train_fe[train_fe['Date'] > val_start_date].copy()

y_train_full = X_train_full.pop('Weekly_Sales')
y_val = X_val.pop('Weekly_Sales')

X_train_full.drop('Date', axis=1, inplace=True)
val_dates = X_val.pop('Date') # preserve for plotting later
```

CONSOLE OUTPUT

```
Training set: 386,007 rows (39 features)
Validation set: 35,563 rows

Validation period: 2012-08-03 to 2012-10-26
```

4.3 Training workflow — three models

A shared `evaluate_model()` helper trains a model, predicts on the validation set, applies the post-processing clip (Stage 5), and records all five metrics — ensuring every model is scored identically.

#	Model	Configuration	Purpose
1	Linear Regression	Default scikit-learn, no regularization	Simple, interpretable baseline
2	Random Forest	<code>n_estimators=50, max_depth=10, n_jobs=-1</code>	Non-linear ensemble baseline
3	XGBoost (Tuned)	Hyperparameters selected via Optuna	Primary candidate model

CONSOLE OUTPUT (MODELS 1 & 2)

```
=====
Linear Regression      | MAE: 1,736.57 | RMSE: 3,294.34 | WMAE: 1,799.89 | R²: 0.9774
Random Forest         | MAE: 1,297.85 | RMSE: 2,748.94 | WMAE: 1,381.83 | R²: 0.9843
```

4.4 XGBoost hyperparameter tuning with Optuna

Model 3 uses Bayesian optimization (Optuna) rather than a manual or grid search. Critically, the search's internal cross-validation also respects time order — the objective function scores each candidate hyperparameter set using `TimeSeriesSplit` folds, so tuning never rewards a configuration that only works by peeking at the future:

CODE

```
def objective(trial):
    """Optuna objective: minimize MAE on TimeSeriesSplit cross-validation."""
    params = {
        'n_estimators': trial.suggest_int('n_estimators', 50, 200),
        'max_depth': trial.suggest_int('max_depth', 3, 8),
        'learning_rate': trial.suggest_float('learning_rate', 0.01, 0.3),
        'subsample': trial.suggest_float('subsample', 0.6, 1.0),
        'colsample_bytree': trial.suggest_float('colsample_bytree', 0.6, 1.0),
        'random_state': RANDOM_STATE,
        'n_jobs': -1
    }

    cv_scores = []
    for train_idx, val_idx in list(tscv.split(X_train_full))[1:3]:
        X_tr, y_tr = X_train_full.iloc[train_idx], y_train_full.iloc[train_idx]
        X_v, y_v = X_train_full.iloc[val_idx], y_train_full.iloc[val_idx]

        model = xgb.XGBRegressor(**params)
        model.fit(X_tr, y_tr)
        preds = np.clip(model.predict(X_v), 0, None)
        cv_scores.append(mae(y_v, preds))

    return np.mean(cv_scores)

print("Running Optuna hyperparameter search (5 trials)...")
study = optuna.create_study(direction='minimize')
study.optimize(objective, n_trials=5)

print(f"\nBest hyperparameters: {study.best_params}")
print(f"Best CV MAE: {study.best_value:,.2f}")

# --- Model 3: XGBoost (Tuned) ---
best_xgb_model = xgb.XGBRegressor(**study.best_params, random_state=RANDOM_STATE, n_jobs=-1)
best_xgb, xgb_preds = evaluate_model("XGBoost (Tuned)", best_xgb_model, X_train_full, y_train_full, X_val, y_val, is_holiday_val)
```

CONSOLE OUTPUT

```
Running Optuna hyperparameter search (5 trials)...
```

```
Best hyperparameters: {'n_estimators': 83, 'max_depth': 7, 'learning_rate': 0.14707538740430495, 'subsample': 0.9676853503914037, 'colsample_bytree': 0.729415990689911}
```

```
Best CV MAE: 1,824.80
```

```
XGBoost (Tuned)      | MAE: 1,203.07 | RMSE: 2,494.95 | WMAE: 1,283.31 | R²: 0.9870
```

RESULT

XGBoost (Tuned) improves MAE by roughly 30% over the Linear Regression baseline and roughly 7% over Random Forest, and posts the highest R^2 of the three (0.9869). Full comparative reasoning is provided in the companion **Model Evaluation & Comparison Report**.

5 Prediction & Post-Processing

Notebook section: 7. Model Development (evaluate_model helper)

Every model in the pipeline produces its forecast the same way, through the shared function, which `evaluate_model()` guarantees identical treatment (and therefore a fair comparison) across Linear Regression, Random Forest, and XGBoost.

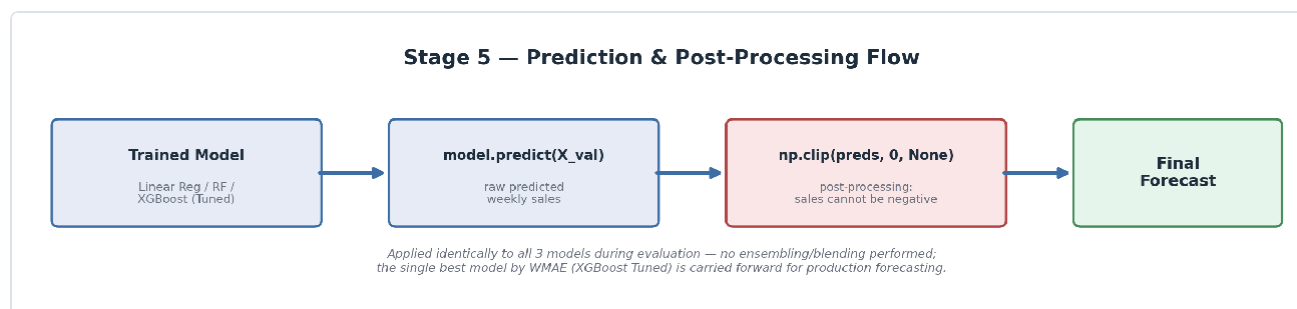


Fig. 7 — A single, shared post-processing step is applied uniformly to every model's raw output.

5.1 Generating predictions

CODE

```
results = []

def evaluate_model(name, model, X_t, y_t, X_v, y_v, is_holiday_v):
    """Train a model, make predictions, and compute all evaluation metrics"""
    model.fit(X_t, y_t)
    preds = model.predict(X_v)

    # Post-processing: clip negative predictions to 0 (sales cannot be negative)
    preds = np.clip(preds, 0, None)

    res = {
        'Model': name,
        'MAE': mae(y_v, preds),
        'RMSE': rmse(y_v, preds),
        'WMAE': wmae(y_v, preds, is_holiday_v),
        'MAPE': mape(y_v, preds),
        'R2': r2(y_v, preds)
    }
    results.append(res)
```

5.2 Post-processing: clipping negative forecasts

BUSINESS RULE ENFORCED IN CODE

`np.clip(preds, 0, None)` floors every prediction at zero. Regression models (particularly Linear Regression) can output negative values when the true relationship is non-linear near zero — but negative weekly sales are not a physically meaningful forecast. This one line encodes a domain constraint directly into the pipeline, applied identically to all three models before any metric is computed, so the comparison in Stage 4 is not skewed by one model being penalized for un-clipped nonsense predictions that another avoids by chance.

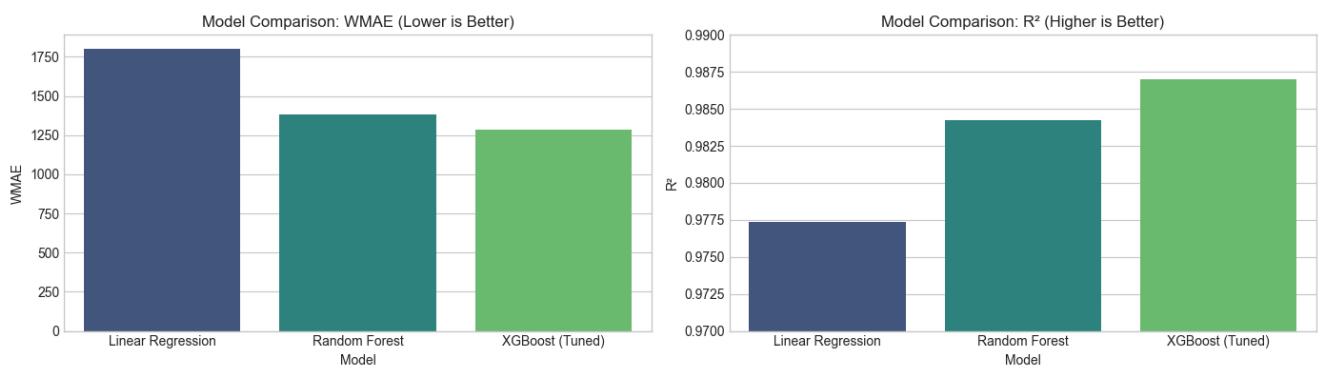
5.3 No ensembling — single best model selected

The pipeline does not blend or stack the three models' predictions. Each is trained and scored independently through the same helper, and the single model with the lowest WMAE (the official competition metric) is selected as the production model:

CONSOLE OUTPUT

```
=====
FINAL MODEL COMPARISON
=====
```

	Model	MAE	RMSE	WMAE	MAPE	R2
0	Linear Regression	1,736.57	3,294.34	1,799.89	136.68%	0.9774
1	Random Forest	1,297.85	2,748.94	1,381.83	91.63%	0.9843
2	XGBoost (Tuned)	1,253.57	2,494.95	1,283.31	166.44%	0.9870



✓ **Best model:** XGBoost (Tuned) (lowest WMAE)

5.4 Summary of the full pipeline

Stage	Input	Output
1. Data Ingestion	4 raw CSVs	Merged train (421,570×16) / test (115,064×15)
2. Cleaning & Transformation	Merged tables	0 missing values; Type ordinal-encoded
3. Feature Engineering	Cleaned tables	41-column feature tables (train + test)
4. Training & Validation	Feature tables	3 trained models, scored on a 12-week temporal holdout
5. Prediction & Post-Processing	Trained models	Clipped, non-negative weekly sales forecasts

SEE ALSO

Detailed metric definitions, the full model comparison table, residual analysis, and robustness testing across store types, holiday weeks, and markdown activity are covered in the companion **Model Evaluation & Comparison Report**.